

队伍编号	MCB2401985
题号	A

基于机器学习的台风路径预测与特征关系分析的综合研究

摘要

台风作为一种强烈的热带气旋，是对人类生活和生产影响巨大的气象现象。台风不仅带来狂风暴雨，还可能引发风暴潮和泥石流等灾害，造成生态破坏和经济损失。因此，研究台风的形成、发展及其路径具有重要意义。本文利用机器学习技术，分析台风的特征参数与环境因素之间的关系，旨在提高台风的预测能力。

针对研究问题，首先分析台风特征参数（如强度、等级、风速）与气温、气压、季风的关系，建立分类评价模型。其次，以 2024 年 9 月的第 13 号台风贝碧嘉为例，探讨其路径预测模型。最后，研究台风登陆后降水量与风速的关系及降水量与距离台风中心的关系。

问题一分析：该题需要构建台风特征参数与环境因子之间的关系模型，并通过建立的模型分析 2024 年 7 月和 9 月的台风数据，评估模型效果。为了解决问题一，本文通过数据预处理、异常值分析等步骤，建立有效的分类评价模型。利用肘部法则确定最优聚类数 $K=6$ ，并采用 KMeans、层次聚类和 DBSCAN 三种聚类算法进行对比分析。结果显示，DBSCAN 在处理复杂模式和噪声方面表现最佳。其次采用爬虫以及现有公开数据集搜集 2024 年台风数据，对爬取到的 2024 年台风数据选取上述搭建的模型进行聚类分析。

问题二分析：利用随机森林、XGBoost 及基于注意力机制的 CNN-LSTM 模型构建台风路径分析预测模型，通过动态时间规整（DTW）算法对比预测路径与实际路径，提升预测的准确性与可靠性。针对问题二，采用 LSTM 模型进行路径预测，结合 DTW 算法评估模型性能，结果表明基于注意力机制的 CNN-LSTM 模型在各项指标上表现优越，均方根误差为 0.0336，预测性能指标达到 0.99533。

问题三分析：研究台风登陆后的降水量与风速的关系，以及降水量与距台风中心的关系，建立相应模型进行分析。针对问题三，建立降水量与风速及距离之间的关系模型，结果显示指数衰减模型在拟合效果上优于幂律衰减模型，为进一步的气象研究提供了可靠依据。

关键词：台风路径预测；机器学习；动态时间规整；指数衰减模型；聚类分析

目录

- 一、 问题重述 1
 - 1.1 问题背景 1
 - 1.2 问题要求 1
- 二、 问题分析 1
 - 2.1 问题一分析 1
 - 2.2 问题二分析 2
 - 2.3 问题三分析 2
- 三、 模型假设 2
- 四、 符号说明 3
- 五、 问题一的模型的建立和求解 3
 - 5.1 数据质量分析 3
 - 5.2 数据预处理 4
 - 5.2.1 数据转换 4
 - 5.2.2 数据清洗 5
 - 5.2.3 特征编码 5
 - 5.2.4 数据标准化 5
 - 5.3 台风分类评价的聚类分析 5
 - 5.3.1 肘部法则 5
 - 5.4 聚类方法的分析与对比 5
 - 5.5 台风数据的斯皮尔曼相关性分析 6
 - 5.5.1 相关性计算 7
 - 5.5.2 热力图分析 7
 - 5.5.3 显著性分析 8
 - 5.6 降维与可视化 8
 - 5.7 台风分类结果地图可视化 9
 - 5.8 2024 年台风数据的聚类分析 10
- 六、 问题二的模型的建立和求解 11
 - 6.1 数据预处理 12
 - 6.1.1 数据清洗与时间特征提取 12

6.1.2 数据标准化	13
6.2 台风路径预测模型建立	13
6.2.1 随机森林模型	13
6.2.2 XGBoost 模型	13
6.2.3 基于注意力机制的 CNN-LSTM 模型	14
6.2.4 模型求解	16
6.2.5 模型训练与优化	17
6.3 动态时间规整 (DTW) 算法	17
6.3.1 DTW 距离计算	17
6.3.2 模型求解	18
七、问题三的模型建立和求解	19
7.1 数据准备	19
7.2 模型建立	19
7.2.1 降水量与风速的关系模型	19
7.2.2 降水量与距离的关系模型	20
7.3 模型评估	20
八、模型的分析与检验	23
8.1 灵敏度分析	23
8.2 误差分析	23
九、模型优缺点分析	24
9.1 DBSCAN 模型	24
9.2 基于注意力机制的 CNN-LSTM 模型	24
9.3 指数衰减模型	24
参考文献	25
A 附录 文件列表	27
B 附录 代码	27

一、问题重述

1.1 问题背景

台风，作为一种强烈的热带气旋，是影响人类生活和生产的重要气象现象。[1] 其不仅带来狂风暴雨，还可能引发风暴潮、泥石流等自然灾害，造成严重的生态破坏和经济损失。因此，研究台风的形成、发展及其运动轨迹，对减轻其带来的灾害具有重要意义。

台风的强度、等级、风速等特征参数与气温、气压、季风等环境因素之间的关系密切。为此，利用大数据分析技术，构建有效的分类和预测模型，能够帮助我们更好地理解台风的动态特征和演变规律，从而提高对台风的预报能力。

通过具体案例分析，如 2024 年 7 月和 9 月的台风情况，参赛者将探讨夏季与秋季台风的异同，并对台风登陆后的风速和降水量变化进行深入研究。这将有助于科学界和决策者更好地应对台风带来的挑战，保护人民的生命财产安全。

1.2 问题要求

问题 1 分析台风特征参数（如强度、等级、风速）与气温、气压、季风的关系，建立分类评价模型，并明确标准。以 2024 年 7 月和 9 月的台风为例，列出台风类别及途经省份，并分析夏台风与秋台风的区别。

问题 2 探讨台风路径的影响因素，建立路径预测模型。以 2024 年 9 月 13 日至 17 日第 13 号台风贝碧嘉为例，预测行进路线，并用动态时间规整（DTW）算法与实际路径对比。

问题 3 研究台风登陆后降水量与风速的关系，以及降水量与距离台风中心的关系。以 2024 年 9 月 16 日至 18 日的第 13 号台风贝碧嘉为例，预测其中心风力和降水量并进行分析。

二、问题分析

2.1 问题一分析

对于问题一，其核心在于利用台风特征参数与环境因子之间的关系建立一个有效的台风分类评价模型。为此，我们需要先找出 2024 年 7 月和 9 月的台风数据。对数据进行质量评估、数据预处理。接着，在将所有数据标准化处理去除量纲后，我们运用随机森林、XGBoost、基于注意力机制的 CNN-LSTM 模型这三种机器学习模型通过对比各模型的选出最优模型。最后，结合动态时间规整（DTW）算法对预测路径与实际路径进行比较，评估模型的预测效果。

2.2 问题二分析

对于问题二，它旨在构建与分析台风路径预测模型，以提升预测的准确性和可靠性。为此，我们需要在相关网站搜集与环境因素相关的数据。对数据进行质量评估、数据预处理。接着，在将所有数据标准化处理去除量纲后，我们运用随机森林、XGBoost、基于注意力机制的 CNN-LSTM 模型这三种机器学习模型通过对比各模型的选出最优模型。最后，结合动态时间规整（DTW）算法对预测路径与实际路径进行比较，评估模型的预测效果。

2.3 问题三分析

对于问题三，在台风登陆后的过程中，风速和降水量会随着距离台风中心的增加而减弱。为了深入了解这一现象，我们需要建立模型来描述中降水量及风速的关系，及降水量与距台风中心距离的关系。接着利用评价指标如 R^2 和 MSE 等指标进行模型评估。最后以 2024 年 9 月 16 日-18 日的第 13 号台风贝碧嘉为例，根据所建立的模型，预测贝碧嘉行进途中的中心风力及降水量，进行分析。

三、模型假设

为简化问题，本文做出以下假设：

- 假设 1 假设所有观测数据之间相互独立，即每个台风的特征参数和环境因素数据不会受到其他台风数据的影响。
- 假设 2 假设选定的特征（如风速、气压、移动方向等）与台风强度存在显著的相关性。
- 假设 3 假设各个特征之间的影响是相对独立的，即每个特征对台风强度的影响可以通过模型捕捉。
- 假设 4：气象数据的质量是可靠的，假设所使用的数据经过充分的预处理和校验，能够反映真实的气象现象。
- 假设 5：台风的运动受自然环境的影响是稳定的，即在分析期间，气候条件和海洋环境变化不会对台风的运动轨迹产生剧烈波动。这一假设有助于减少外部环境因素对模型预测的干扰。

四、符号说明

表 1 符号与说明

符号	单位	说明
d_i	-	第 i 个样本的秩次差异
n	-	样本的数量
x_i	-	时间序列的第 i 个值
y_j	-	时间序列的第 j 个值
t_i	-	第 i 个时间戳
t_j	-	第 j 个时间戳
P_0	mm/h	风速为零时的降水量
γ	-	风速对降水量的敏感程度
P'_0	mm/h	特定条件下的基准降水量
δ	-	风速对降水量影响的强度
P	mm	降水量
V	m/s	风速
D	km	距离
P''_0	mm/h	指数衰减模型初始降水量
γ'	-	指数衰减模型的衰减系数
P'''_0	mm/h	幂律衰减模型初始降水量
δ'	-	幂律衰减模型的指数

五、问题一的模型的建立和求解

5.1 数据质量分析

数据质量分析是数据挖掘过程中不可或缺的环节，它是数据预处理的前提，也是数据挖掘分析结论有效性和准确性的基础。因此，进行科学、客观的数据质量评估是非常必要而且十分重要的。本文针对网盘 [2] 中的数据质量分析如下：

相较于使用 3σ 准则剔除异常值，箱形图不需要数据服从正态分布。借助箱型图，我们能够非常直观地发现异常值并了解数据分布的情况。由于数据集数据不服从正态分布，因此我们选择箱形图进行质量评估。

首先，我们将数据按指标划分，分别进行从小到大的排序；然后计算出各指标数据的上四分位数（Q3）、下四分位数（Q1）和四分位距（IQR），四分位距与上、下四分位数的关系如式（1）所示：

$$IQR = Q3 - Q1 \quad (1)$$

随后，我们计算出上限和下限，上限和下限的计算公式为：

$$\text{上限} = Q3 + 1.5 \times IQR \quad (2)$$

$$\text{下限} = Q1 - 1.5 \times IQR \quad (3)$$

最后，判断异常值：超出上限和下限的数值都被认为是异常值。我们根据 1945-2023 年的台风数据所绘制的箱型图。**图1a**和**图1b**展示了数据预处理前后的箱型图，通过对比可以明显观察到异常值的变化情况。

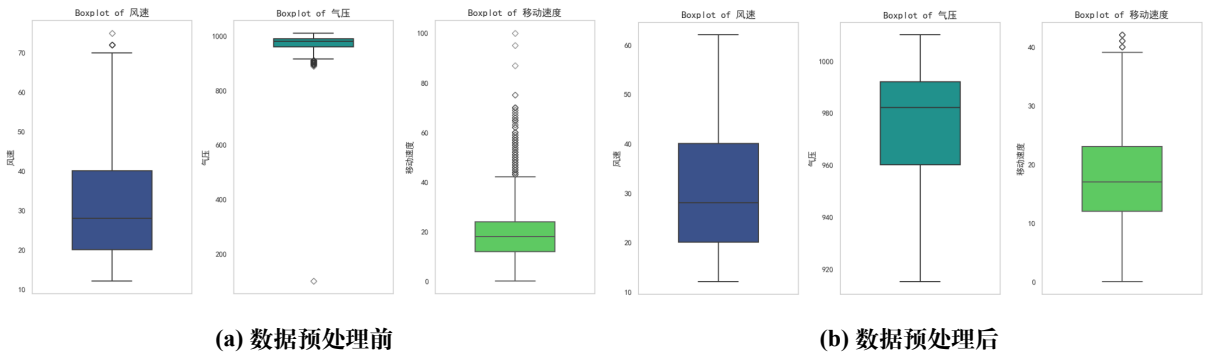


图 1 箱型图

5.2 数据预处理

数据预处理是数据分析和机器学习中的关键步骤，其目的是清理、转换和准备数据，以便后续的建模和分析。预处理过的数据集将会大幅度提高后续建立模型的性能和模型结果的准确性，我们将根据数据质量分析情况开展相应数据预处理工作。本文对收集的台风数据进行的数据预处理过程如下：

5.2.1 数据转换

首先，我们对台风相关的时间列进行日期时间格式的转换，确保了数据的一致性。接着，通过计算台风起始时间与结束时间间隔，我们得到了每个台风的持续时间（以天为单位）。这一特征对于后续分析台风的影响至关重要。最后，从当前台风的时间中提取出月份信息，通过定义函数，我们将月份转换为季节标签（夏台风或秋台风）。根据季节分类的结果，仅保留夏台风和秋台风的数据，去除了无关项。

5.2.2 数据清洗

在数据分析中，缺失值可能会影响模型的训练和预测效果。通过删除含有缺失值的行，确保后续分析的数据完整性。此步骤有效提高了数据的质量，减少了噪声干扰，我们删除含有缺失值的行。

5.2.3 特征编码

特征编码是将类别数据转化为数值数据的重要步骤。通过编码函数，我们将台风强度和移动方向转换为数值型数据；与此同时，利用映射关系，将台风强度和移动方向转换为对应的数字，便于模型的训练与分析。

5.2.4 数据标准化

为了消除量纲影响，我们需要对数据进行标准化处理。标准化的公式如下：

$$X'_{ij} = \frac{X_{ij} - X_{\min,j}}{X_{\max,j} - X_{\min,j}} \quad (4)$$

式中， X'_{ij} 是标准化后的数值， X_{ij} 是指标体系中第 j 个指标中的第 i 个数值， $X_{\min,j}$ 和 $X_{\max,j}$ 分别是第 j 个指标的最小值和最大值。

5.3 台风分类评价的聚类分析

5.3.1 肘部法则

我们使用肘部法则这一常用方法来确定最优聚类数。通常情况下，样本划分的类别数越多，每一类内部的结构越松散，聚合系数（例如轮廓系数）越小；既而反之 [3] 根据肘部法则，样本所划分的类别数越多，聚合系数通常会下降。基于样本数据所得到的聚合系数与聚类类别数的关系图如 **图2** 所示。可以观察到聚类类别数从 1 到 6 时，聚合系数的下降幅度较大；而当类别数大于 6 时，聚合系数的变化幅度逐渐趋于平缓。因此，我们确定肘部位置为 $K = 6$ ，即最佳聚类数为 6。

5.4 聚类方法的分析与对比

根据肘方法 [4] 选择的合适聚类数目为 $K = 6$ ，我们使用 KMeans[5]、层次聚类 (Agglomerative Clustering) [6] 和 DBSCAN [7] 三种聚类算法对台风数据进行聚类分析，并采用轮廓系数、Calinski-Harabaz 指数和 Davies-Bouldin 指数对结果进行评价。聚类算法评价指标结果如**表2**。

KMeans 通过迭代优化每个簇的质心来形成 K 个簇，适合处理大规模数据然而，由于其对初始质心敏感，可能在复杂模式下无法准确捕捉台风数据中的细微差异。相较之

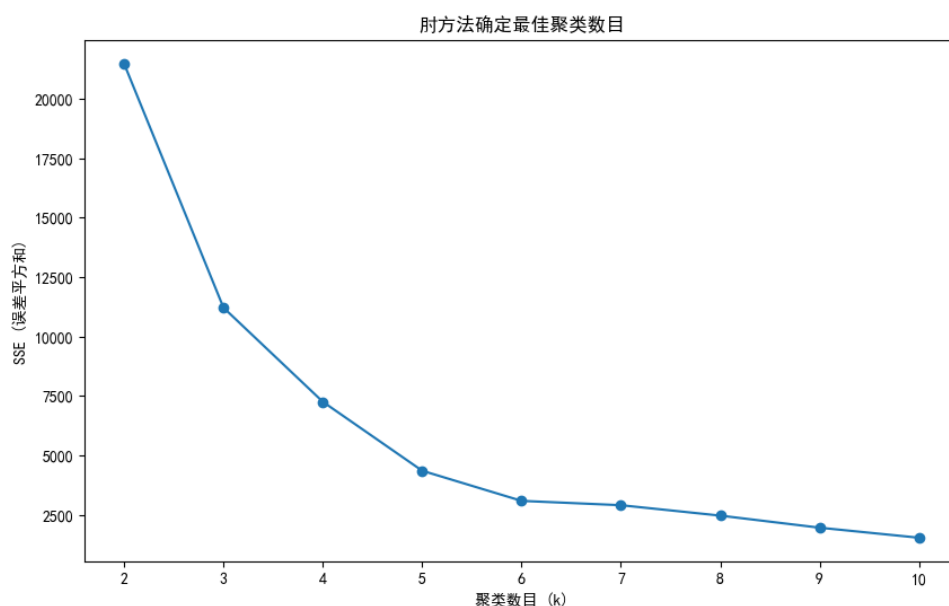


图 2 聚合系数与聚类类别数关系图

下，层次聚类采用自底向上的方式，能够生成树状图，直观展示数据的层次结构，适合于分析台风特征之间的关系，尽管在处理大数据时计算复杂度较高。DBSCAN 则基于密度来形成聚类，能够发现任意形状的簇，并有效处理噪声，这对于台风分类问题尤为重要，因为台风数据中常存在不规则的模式和异常值。通过综合评估这三种算法的聚类效果，我们能够在复杂的气象数据中提取出有效信息，提升台风分类的准确性。

表 2 聚类算法评价指标

聚类算法	轮廓系数 (SC)	Calinski-Harabaz 指数 (CH)	Davies-Bouldin 指数 (DB)
KMeans	0.654891	92430.456525	0.414592
层次聚类 (Agglomerative Clustering)	0.654891	92430.456525	0.414592
DBSCAN	0.644629	109302.072225	0.480731

聚类效果评估指标主要包括轮廓系数 [8]、Calinski-Harabaz 指数 [9] 和 Davies-Bouldin 指数 [10]。综合对比轮廓系数、Calinski-Harabaz 指数和 Davies-Bouldin 指数的结果，DBSCAN 在此次聚类分析效果最佳。它的轮廓系数和 Calinski-Harabaz 指数均较高，而 Davies-Bouldin 指数最低，说明其在簇内紧密性和簇间分离度方面表现更优。

5.5 台风数据的斯皮尔曼相关性分析

本文对台风数据中的四个关键特征：台风强度、台风等级、风速和气压进行了斯皮尔曼相关性分析。通过计算这些特征之间的相关性矩阵，我们可以了解它们之间的相互

关系，并为后续的聚类分析提供依据。

我们首先从 Excel 文件中加载台风数据，并选择以下特征台风强度、台风等级、风速、气压进行分析。在数据预处理阶段，我们已经对缺失值进行处理，确保了用于相关性分析的数据完整性。

5.5.1 相关性计算

斯皮尔曼相关系数（Spearman's rank correlation coefficient）[11] 是一种非参数统计方法，用于评估两个变量之间的单调关系，其计算公式如??本文利用斯皮尔曼相关性方法计算特征之间的相关性矩阵及其显著性水平（p 值）。

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (5)$$

其中， d_i 是第 i 个样本在两个变量中的秩次差异; n 是样本的数量。

5.5.2 热力图分析

为了更直观地展示台风强度、等级、风速和气压之间的相关性，我们使用热力图可视化斯皮尔曼相关性矩阵。热力图能够通过颜色的深浅反映变量之间的相关性强度，从而快速识别出特征之间的关系，暖色表示正相关，冷色表示负相关。

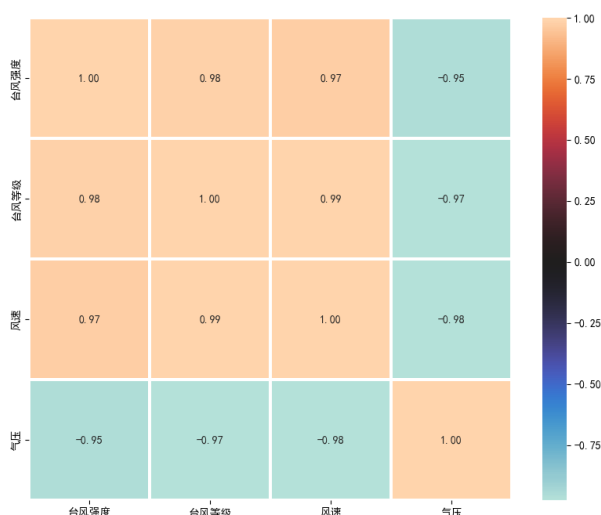


图3 斯皮尔曼相关性矩阵热力图

从热力图中可以明显看出，台风强度与台风等级、台风风速之间的相关性接近 1，表明它们之间存在显著正相关关系，这意味着风速随着台风等级的提高而显著增加。此外，气压与其他特征（尤其是风速和台风等级）之间的负相关性也很强，表明较强的台风通常伴随较低的气压和较高的风速。

5.5.3 显著性分析

p-value 相关性是否具有统计显著性是统计假设检验中的一个关键指标。通过样本的斯皮尔曼相关系数 ρ ，计算相应的检验统计量（类似于 t 检验中的 t 值）。对于大样本，检验统计量可以通过以下公式近似：

$$t = \rho \times \sqrt{\frac{n-2}{1-\rho^2}}$$

其中， ρ 是样本的斯皮尔曼相关系数； n 是样本的大小。

表 3 显示了所有相关性计算的 p 值均为 0.0，这些相关性结果在统计上都是显著的 (p 值 < 0.05)。具体来说，p 值小于 0.05 的条件被标记为 True。表 4 显示了显著性标记结果。

表 3 斯皮尔曼相关性显著性水平 (p 值)

	台风强度	台风等级	风速	气压
台风强度	0.0	0.0	0.0	0.0
台风等级	0.0	0.0	0.0	0.0
风速	0.0	0.0	0.0	0.0
气压	0.0	0.0	0.0	0.0

表 4 显著性标记 (p 值 < 0.05)

	台风强度	台风等级	风速	气压
台风强度	True	True	True	True
台风等级	True	True	True	True
风速	True	True	True	True
气压	True	True	True	True

通过对台风强度、等级、风速和气压四个特征之间的相关性分析，进一步说明这些特征之间存在非常显著的相关性，特别是风速和台风等级之间的相关性接近完美，同时气压与其他特征之间的强负相关性符合物理常识。

5.6 降维与可视化

主成分分析 (PCA) [12] 是一种常用的降维技术，其核心思想是通过线性变换，将高维数据投影到低维空间，它能够有效地减少特征维度，同时尽可能保留数据的变异

性。本文将台风的四个特征：台风强度、台风等级、风速和气压应用 PCA 进行降维，旨在获得一个可以清晰展示不同台风类别的数据表示。

我们首先要对数据进行标准化，从而确保每个特征对降维过程的影响相对均衡，以消除不同特征量纲的影响。接着使用 PCA 将数据降维至二维空间。通过提取两个主成分，我们能够最大程度地保留数据的变异性。这一过程使我们得以将高维特征映射到易于可视化的低维空间。

为了直观展示降维后的数据分布，我们绘制了 2D 散点图。如图4所示，每个点代表一个台风样本，并根据其类别进行颜色标记。散点图中的第一主成分映射到 x 轴，第二主成分映射到 y 轴，不同颜色则表示不同的台风类别。

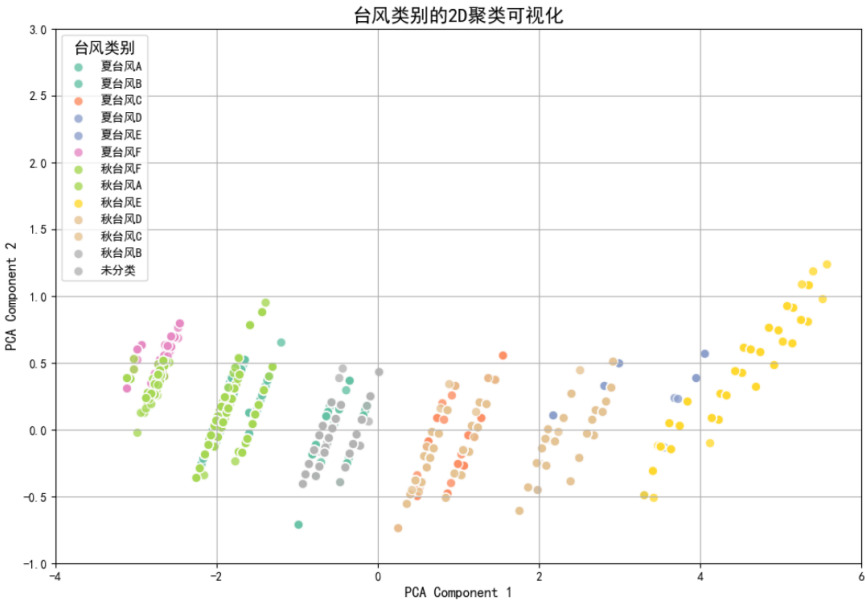


图 4 台风类别的 2D 聚类可视化

通过 2D 散点图，我们可以观察到不同台风类别在降维后的特征空间中的分布情况。不同台风类别在图中呈现出明显的分散与聚集现象；某些台风类别聚集得较为紧密，表明它们在特征空间中具有相似的特征；而其他类别则可能存在较大的重叠或分散。通过分析每个类别的具体特征，我们可以进一步探讨其对台风强度、等级、风速及气压的影响。

5.7 台风分类结果地图可视化

为了更直观地展示不同台风类别的地理分布，我们使用地图可视化技术将台风的经纬度数据映射到地球表面。通过这种方式，我们可以更清晰地了解台风在全球范围内的分布情况。首先我们从台风分类结果中提取了经度、纬度和台风类别等相关信息。这些信息为我们的地图可视化提供了基础数据。接着，我们使用 Basemap 库绘制了台风类别的地理分布图。绘制的地图如图5所示。从图5中我们可得，不同类别的台风在地图上呈

现出不同的分布模式，某些类别的台风在特定区域更加集中，反映了其活动的季节性和地理特征。通过颜色的区分，可以直观观察到不同类别台风的活动区域。

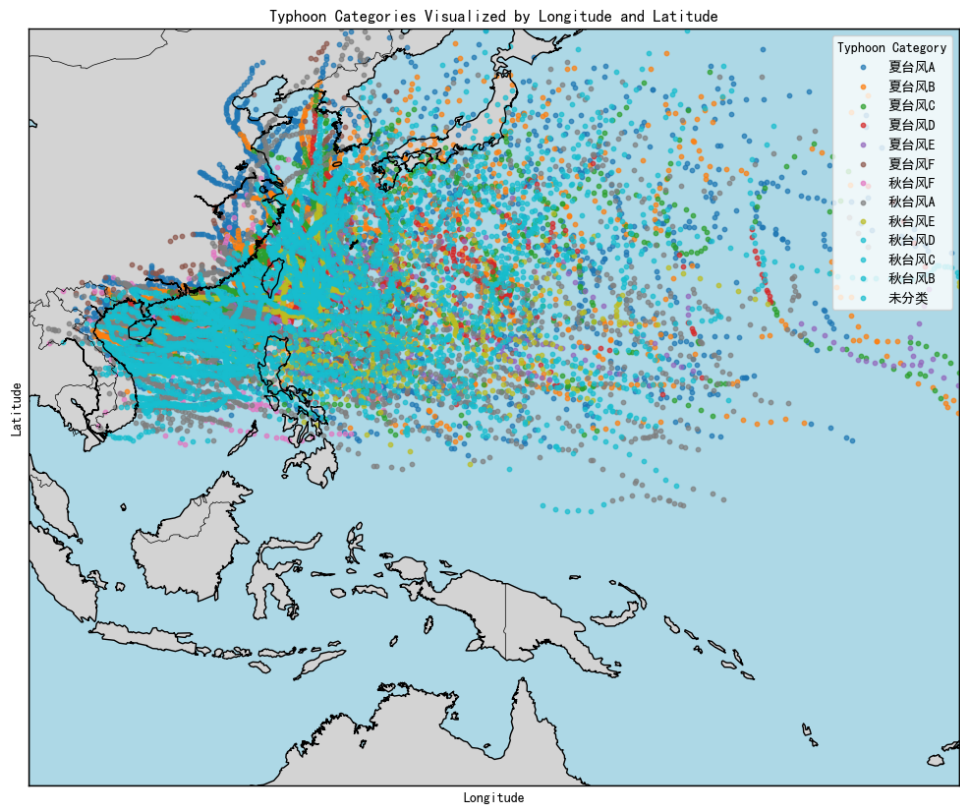


图 5 台风类别的地理分布图

5.8 2024 年台风数据的聚类分析

为了揭示了不同台风类别之间的潜在关系和特征，我们通过聚类分析方法对 2024 年的台风数据进行深入研究。

首先，从原始数据中提取出相关特征，包括台风强度、风速和气压。我们对日期字段进行处理，提取出月份信息，并筛选出 7 月和 9 月的台风数据。为了更好地进行分类，我们定义了夏台风为 6 月至 8 月的台风，其特点是一般强度较高，风速快，影响区域广泛，多发生在沿海省份，影响海洋和陆地；定义秋台风为 9 月至 11 月的台风，其通常风速较夏台风稍弱，但依然具有显著影响。可能受到冷空气的影响，路径更为复杂，可能造成更大的降水。接着，我们对台风强度进行编码，以便于后续分析。编码标准如下：

表 5 台风强度编码标准

台风强度	热带低压	热带风暴	强热带风暴	台风	强台风	超强台风
编码	1	2	3	4	5	6

为了分析台风的特征，我们对选择的特征进行标准化处理。然后，使用 DBSCAN

算法进行聚类分析。该算法具有较强的噪声容忍能力，能够有效识别出不同的聚类。根据聚类结果，我们为每个聚类分配了一个类别标签，例如“夏台风 A”、“秋台风 B”等。

为了可视化聚类结果，我们使用 PCA 将高维特征降维至二维空间。PCA 能够有效地提取数据中的主要成分，并减少维度，帮助我们更直观地观察数据的分布情况。图6展示了不同台风类别的 2D 聚类可视化。散点图中，每个点代表一个台风样本，颜色表示其所属的聚类类别。通过该图，我们可以观察到台风类别在特征空间中的分布情况，识别出各类别之间的相似性和差异性。

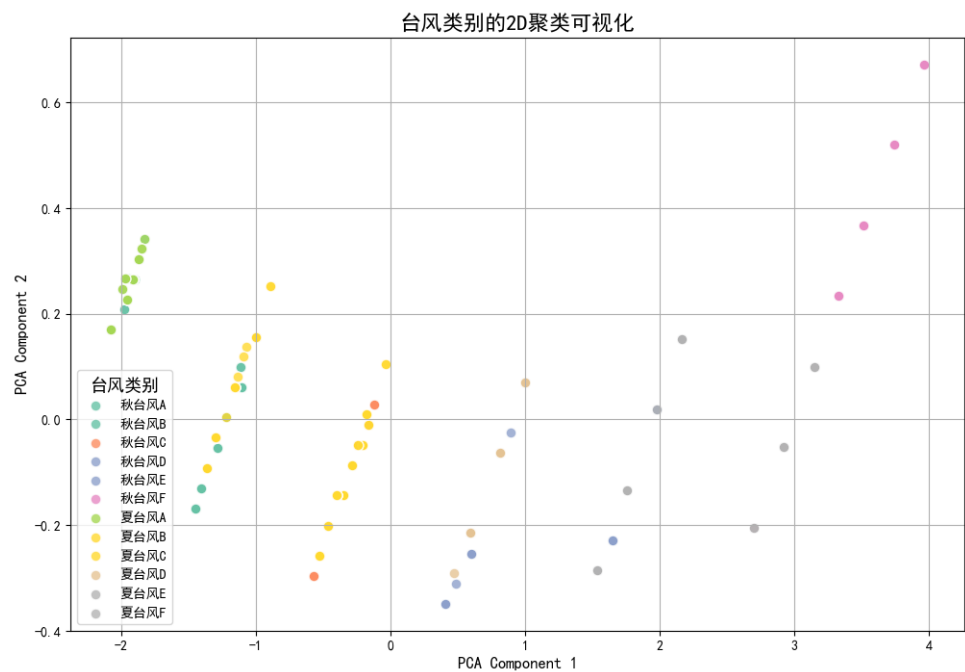


图 6 2024 年台风类别的 2D 聚类可视化

在 2024 年 7 月和 9 月的台风数据中，我们可以看到不同的台风类别，以及它们经过的省份。

通过以上表格和分析，可以清楚地看到夏台风和秋台风的类别及其影响省份的区别。夏台风通常伴随更强的风速和影响范围，而秋台风则可能因季节变化和冷空气的影响，导致路径复杂和降水变化。在气象预测和防灾减灾方面，对这两类台风的区别理解非常重要。

六、问题二的模型的建立和求解

本文研究了台风路径预测模型的构建与分析，旨在通过考虑多种环境因素对台风路径的影响，提升预测的准确性与可靠性。我们采用循环神经网络（LSTM）作为主要建模工具，并通过动态时间规整（DTW）算法与实际路径进行比较，评估模型的预测效果。

表 6 台风类别及途经省份

月份	台风类别	途经省份
7 月夏台风	夏台风 A	湖北省、湖南省
	夏台风 B	江西省、湖北省
	夏台风 C	海南省、福建省
	夏台风 D	福建省
	夏台风 E	台湾省
	夏台风 F	台湾省
9 月秋台风	秋台风 A	江苏省、安徽省、河南省
	秋台风 B	上海市、江苏省、安徽省
	秋台风 C	浙江省、安徽省、江苏省
	秋台风 D	上海市、江苏省
	秋台风 E	上海市
	秋台风 F	海南省

6.1 数据预处理

在台风路径预测模型的建立过程中，高质量的数据不仅可以提升模型的准确性，还能增强模型的泛化能力。本文使用了台风数据集，数据来源于网盘和。数据包含了台风的多种特征，如强度、等级、风速、气压等，以及台风的时间信息。这些信息为后续的特征提取和模型训练提供了基础。

6.1.1 数据清洗与时间特征提取

我们提取了与台风路径预测密切相关的特征，如台风强度、台风等级、风速、气压、移动方向、移动速度；时间特征，如台风的起始时间、结束时间和当前时间。为了确保数据的完整性，我们对数据进行了过滤，去除了缺失值且将时间特征转换为数值格式。通过对时间特征的处理，我们生成了新的特征：当前时间的小时、当前时间的月份、台风的持续时间（以天为单位）、已持续时间（以天为单位）。计算公式如下：

$$\text{持续时间_天} = \frac{(\text{台风结束时间} - \text{台风起始时间})}{86400} \quad (6)$$

$$\text{已持续时间_天} = \frac{(\text{当前台风时间} - \text{台风起始时间})}{86400} \quad (7)$$

此外，在特征集中，某些特征可能是非数值型的，例如移动方向。我们使用因子化的方法将其转换为数值型，以便用于模型训练。最终构建了完整的特征集 X 和目标集

y 。

6.1.2 数据标准化

多源数据需在时间和空间上进行对齐，确保每个时间点都有对应的环境特征。我们使用最近邻插值法确保各时间点的特征一致性。为了提高模型的预测性能，构建了多个与台风路径相关的特征，包括：

- 气象特征：气温、气压、风速、洋流。
- 时间特征：台风生成时间、持续时间。
- 空间特征：台风当前经纬度。

接着使用 Z-score 标准化方法对特征进行处理，以消除不同量纲对模型训练的影响：

$$Z = \frac{X - \mu}{\sigma} \quad (8)$$

其中， X 为原始特征值， μ 为特征均值， σ 为特征标准差。

6.2 台风路径预测模型建立

在本研究中，我们采用了多种机器学习模型来进行台风路径的预测。这些模型包括随机森林 [13]、XGBoost[14] 和基于注意力机制的 CNN-LSTM[15] 三种模型，每种模型都有其独特的优势和适用场景。

6.2.1 随机森林模型

随机森林是一种基于多个决策树的集成学习监督式算法。[13] 该方法在回归分析中能够高效识别显著的解释变量，并比较其对因变量的贡献率，从而为后续建模提供参考。随机森林计算特征变量重要性分数的公式如下：

$$I(X_j) = \frac{1}{n} \sum_{i=1}^n (E_{ji} - E_i) \quad (9)$$

其中， X_j 是第 j 个特征变量， $I(X_j)$ 是其重要性分数， E_i 是第 i 个决策树的袋外误差， E_{ji} 是在特征 X_j 变更后的新的袋外误差。通过计算各特征的重要性分数，可以选择出对预测效果影响最大的特征。

6.2.2 XGBoost 模型

XGBoost（极限梯度提升算法）是一种基于梯度提升实现的机器学习算法。[14] 它以 CART 回归树和随机森林为基础，具备更好的扩展性和训练速度。为了克服传统损失函数的过拟合缺点，该模型引入了正则化函数：

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \|\omega\|^2 \quad (10)$$

其中, γ 和 λ 是正则项的惩罚系数, T 是回归树的叶子节点个数, ω 是叶子节点的权重。对于训练样本 $G = \{(x, y_1), (x, y_2), \dots, (x, y_m)\}$, XGBoost 模型的目标函数可表示为:

$$O^{\text{XGBoost}}(f_k) = \sum_{i=1}^n l(y_i, \hat{y}_i^{t-1} + f_k(x_i)) + \Omega(f_k) \quad (11)$$

其中, l 是损失函数, $\hat{y}_i^{t-1}$ 为第 $t-1$ 轮的预测值。通过优化目标函数, 可以有效提高模型的预测能力。

6.2.3 基于注意力机制的 CNN-LSTM 模型

在本研究中, 我们采用基于注意力机制的 CNN-LSTM 模型进行台风路径预测。该模型结合了卷积神经网络 (CNN)、长短期记忆网络 (LSTM) 和注意力机制, 以有效提取时序数据的局部特征并捕捉序列中的长距离依赖关系, 从而提高预测的准确性。[15]

CNN 通过卷积操作提取输入数据的特征。给定一个输入序列 $X = [x_1, x_2, \dots, x_n]$, 卷积操作可以表示为:

$$Z_i = (X * K)(i) = \sum_{j=1}^k x_{i+j-1} \cdot k_j \quad (12)$$

其中, K 是卷积核, Z_i 是卷积操作的输出, k 是卷积核的大小。通过多个卷积层, 可以逐步提取更高层次的特征。

接下来, 提取到的特征输入 LSTM 层, 以捕捉时间上的依赖关系。LSTM 的状态更新可以表示为:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{遗忘门}) \quad (13)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{输入门}) \quad (14)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (\text{候选记忆单元}) \quad (15)$$

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \quad (\text{记忆单元更新}) \quad (16)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{输出门}) \quad (17)$$

$$h_t = o_t \cdot \tanh(C_t) \quad (\text{隐藏状态输出}) \quad (18)$$

在此基础上, 引入注意力机制来增强模型对关键时刻信息的关注。给定隐藏状态 h_t 和上下文向量 c_t , 注意力权重可以通过以下公式计算:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^T \exp(e_j)} \quad (19)$$

其中, $e_t = v_a^T \cdot \tanh(W_a \cdot h_t + U_a \cdot c_{t-1})$ 是注意力得分, v_a 、 W_a 和 U_a 是学习的参数。注意力权重 α_t 用于加权求和, 从而生成上下文向量:

$$c_t = \sum_{t=1}^T \alpha_t h_t \quad (20)$$

最终, LSTM 的输出结合上下文向量进行预测:

$$\hat{y}_t = \text{Predictor}(h_t, c_t) \quad (21)$$

此模型结构能够充分利用台风数据的空间特征和时间特征, 同时通过注意力机制增强对重要信息的聚焦, 从而提高预测的准确性和模型的有效性。图 7 展示了 CNN-LSTM 模型的整体结构。

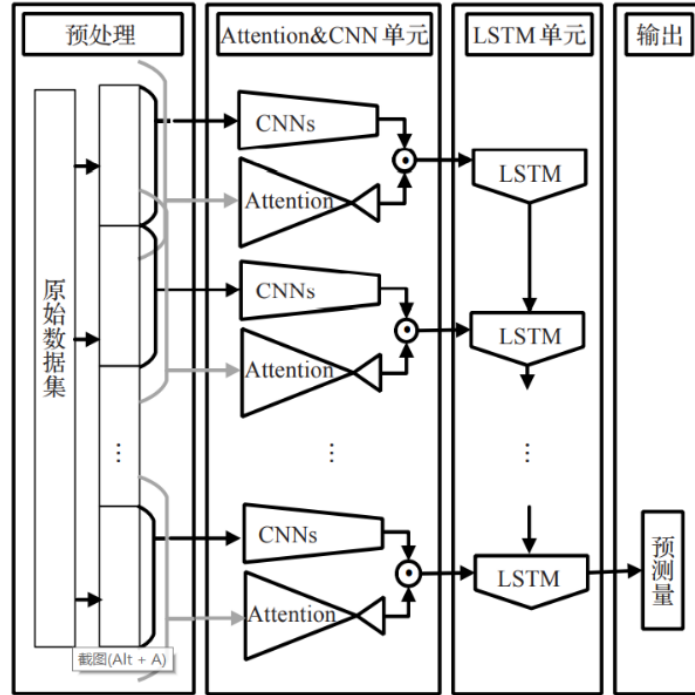


图 7 基于注意力机制的 CNN-LSTM 模型结构

图 8 展示了评价数据质量并提取与台风相关的关键数据项的重要性分析过程。



图 8 构建台风路径预测模型流程图

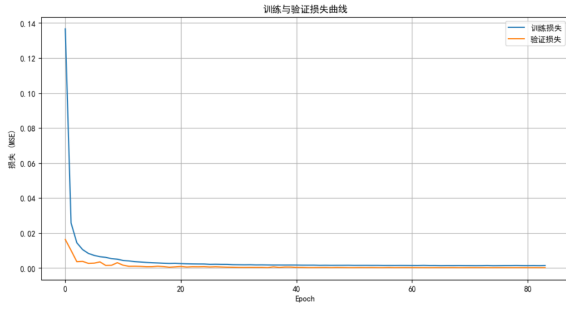
6.2.4 模型求解

在本研究中，我们分别采用了随机森林、XGBoost 和基于注意力机制的 CNN-LSTM 三种模型进行计算，并评估其预测性能。为了全面比较这些模型的效果，我们计算了均方根误差（RMSE）[16]、平均绝对误差（MAE）[17]、均方误差（MSE）[18] 和预测性能指标。结果如表 7 所示：

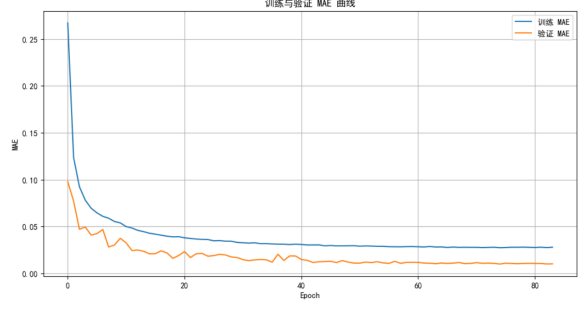
表 7 预测精度的统计特征

统计指标	随机森林	XGBoost	CNN-LSTM
RMSE	0.1154	0.0811	0.0336
MAE	0.0774	0.0555	0.0295
MSE	0.0134	0.0066	0.0071
预测性能	0.63027	0.86857	0.99533

从表 7 中可以看出，基于注意力机制的 CNN-LSTM 模型在各项指标上表现最佳，其均方根误差（RMSE）为 0.0336，平均绝对误差（MAE）为 0.0295，均方误差（MSE）为 0.0071，预测性能指标达到 0.99533。其训练与验证损失曲线、训练与验证 MAE 曲线如图 9b 和图 9a 所示。这些结果表明，基于注意力机制的 CNN-LSTM 模型相较于其他模型具有更低的误判率和更高的预测精度。因此，我们选择基于注意力机制的 CNN-LSTM 模型作为构建台风路径预测模型的主要方法。



(a) 训练与验证损失曲线



(b) 训练与验证 MAE 曲线

图 9 基于注意力机制的 CNN-LSTM 模型的相关数据

6.2.5 模型训练与优化

模型训练过程中，使用交叉验证法对模型进行评估，调节超参数以优化模型性能。模型的超参数包括 LSTM 层数、学习率、批大小等。采用网格搜索法进行超参数调节，通过交叉验证选择最佳参数组合：

$$\text{最优参数} = \arg \min_{\theta} \left(\frac{1}{k} \sum_{j=1}^k \text{损失}(j, \theta) \right) \quad (22)$$

6.3 动态时间规整 (DTW) 算法

动态时间规整 (DTW) 是一种广泛用于计算两个时间序列相似度的算法，尤其适用于在不同时间尺度下的数据对齐。[19] DTW 通过允许时间序列在时间轴上进行非线性变形，从而提高了对不同长度和速率的序列的比较能力。在本研究中，DTW 算法用于评估预测路径与实际路径之间的相似度，以验证模型的预测效果。

6.3.1 DTW 距离计算

DTW 距离的计算基于递归的方式，公式如下：

$$DTW(i, j) = \text{cost}(i, j) + \min \begin{cases} DTW(i-1, j) \\ DTW(i, j-1) \\ DTW(i-1, j-1) \end{cases} \quad (23)$$

其中， $\text{cost}(i, j)$ 为点 (i, j) 的距离，通常采用欧几里得距离计算：

$$\text{cost}(i, j) = \sqrt{(x_i - x_j)^2 + (t_i - t_j)^2} \quad (24)$$

这里， x_i 和 y_j 分别表示时间序列的值， t_i 和 t_j 表示相应的时间戳。DTW 算法通过动态规划技术构建一个累计距离矩阵，进而计算出从序列 1 到序列 2 的最优对齐路径。

在本次实验中，DTW 距离为 7.3179。这一数值表明预测路径与实际路径之间的相似性程度。较小的 DTW 距离通常意味着预测路径与实际路径的对齐效果较好，模型在时间序列的不同阶段具有较高的预测能力。具体而言，7.3179 的 DTW 距离反映了模型在捕捉台风路径变化方面的有效性。

DTW 算法的路径可视化展示了时间序列的对齐情况。通过可视化，我们可以清晰地看到预测路径与实际路径在时间轴上的对应关系，从而评估模型的准确性和有效性。如图 10 所示，DTW 路径图中，横轴和纵轴分别表示实际路径和预测路径的时间序列值。路径的弯曲程度反映了模型在不同时间段的预测能力。可视化结果有助于进一步分析模型在特定时间段的预测表现。

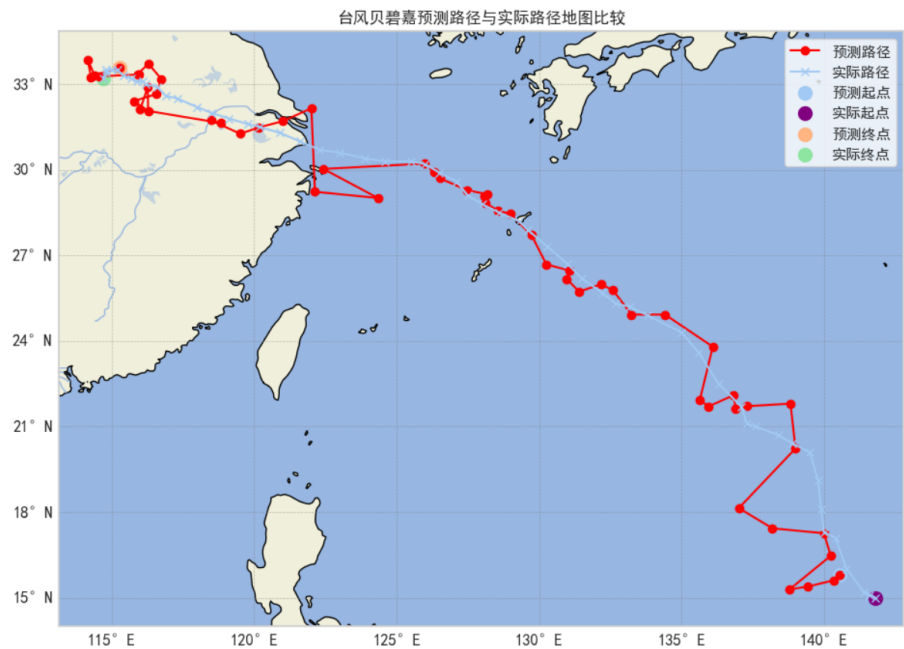


图 10 台风贝碧嘉预测路径与实际路径比较

本文使用 DTW 算法来比较预测路径与实际路径之间的相似度。通过计算 DTW 距离，我们能够量化预测模型的表现。DTW 的优势在于其处理变速、非线性变化的能力，使其在复杂的时序数据分析中表现尤为出色。此外，DTW 的结果可以为模型改进提供反馈，帮助优化预测算法。

6.3.2 模型求解

通过上图可知，台风路径的预测值与实际值在某些时间点上较为接近。2024 年 9 月 13 日-17 日每日 14 点的第 13 号台风贝碧嘉的中心位置如下结果（表 1 所示）。

表 8 2024 年 9 月 13 日-17 日每日 14 点的第 13 号台风贝碧嘉的中心位置

时间	台风中心位置预测值 (经度/纬度)	台风中心位置真实值 (经度/纬度)
13 日 14:00	136°09'89"E, 23°79'73"N	135°6'E, 23°6'N
14 日 14:00	130°25'87"E, 26°68'29"N	130°3'E, 27°3'N
15 日 14:00	126°51'76"E, 29°70'41"N	126°2'E, 30°1'N
16 日 14:00	120°14'57"E, 31°47'92"N	120°5'E, 31°5'N
17 日 14:00	116°55'62"E, 32°65'05"N	116°3'E, 33°3'N

七、问题三的模型建立和求解

7.1 数据准备

在本研究中，数据的准备工作是至关重要的。数据主要来源于国家气象局 [20]、中国气象局热带气旋资料 [21] 和星云图开放平台 [3]，数据涵盖了台风、降水及风速的相关信息。同样的，我们进行了数据质量评估和数据预处理。

7.2 模型建立

为了研究降水量与风速及距离之间的关系，我们选择了两种模型：指数衰减模型和幂律衰减模型。我们分别建立了降水量与风速和降水量与距离的关系模型。

7.2.1 降水量与风速的关系模型

线性模型

首先，建立降水量 P 与风速 V 之间的线性回归模型，形式为：

$$P = \beta_0 + \beta_1 V + \epsilon \quad (25)$$

其中， β_0 是截距， β_1 是风速的回归系数， ϵ 是误差项。

非线性模型

在非线性模型方面，我们考虑了指数衰减模型和幂律衰减模型。降水量 P 与风速 V 之间的指数衰减模型表示为：

$$P = P_0 \cdot e^{-\gamma V} \quad (26)$$

幂律衰减模型则为：

$$P = P'_0 \cdot V^{-\delta} \quad (27)$$

其中， P_0 、 γ ； P'_0 、 δ 是指数衰减模型中的常数。分别表示当风速为零时的降水量、风速对降水量影响的敏感程度；在特定条件下的基准降水量、风速对降水量影响的强度。

7.2.2 降水量与距离的关系模型

线性模型

为了建立降水量与距离之间的关系模型，我们利用 Haversine 公式 [22] 计算台风登陆点与各气象站之间的距离。Haversine 公式的数学表达为：

$$d = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right) \quad (28)$$

其中， R 为地球半径， ϕ_1 和 ϕ_2 为两个地点的纬度， $\Delta\phi$ 和 $\Delta\lambda$ 为两地的纬度和经度差。

在计算距离后，我们建立降水量与距离之间的线性回归模型，形式为：

$$P = \beta_0 + \beta_1 d + \epsilon \quad (29)$$

非线性模型

同样，我们建立非线性的模型。降水量 P 与距离 D 之间的指数衰减模型表示为：

$$P = P_0'' \cdot e^{-\gamma' D} \quad (30)$$

幂律衰减模型则为：

$$P = P_0''' \cdot D^{-\delta'} \quad (31)$$

其中， P 为降水量， D 为距离。 P_0'' 是指数衰减模型的初始降水量、 γ' 是指数衰减模型的衰减系数、 P_0''' 是幂律衰减模型的初始降水量、 δ' 是幂律衰减模型的指数。

7.3 模型评估

我们对降水量与风速及距离的关系模型进行了评估，使用 R^2 和均方误差（MSE）作为评价指标。

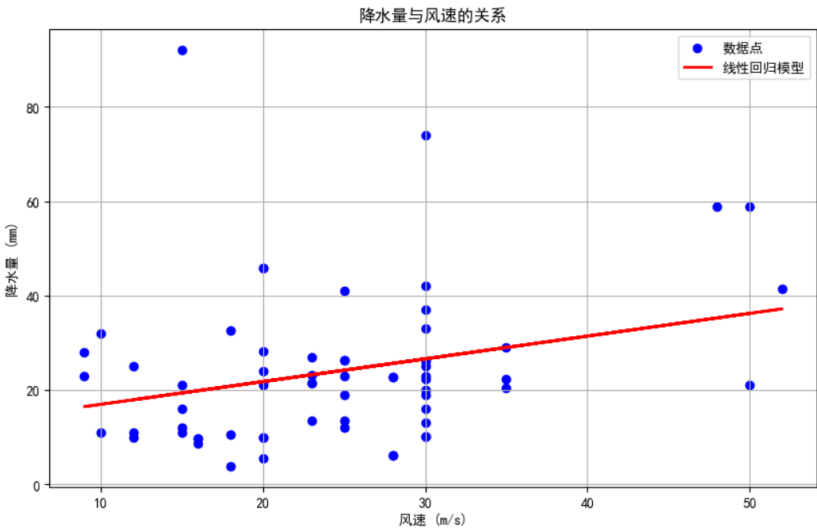
降水量与风速的关系模型评估结果总结如表9所示。

表 9 降水量与风速模型评估结果

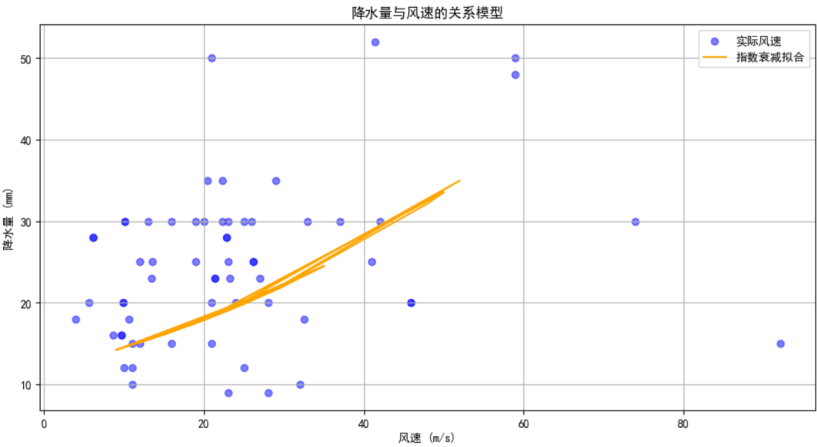
模型类型	R^2	MSE
线性回归模型	0.0800	240.77
指数衰减模型	0.1000	0.3607
幂律衰减模型	0.0667	0.3741

进一步，我们使用散点图和回归线对线性模型和非线性模型进行可视化，图 11 所示。从图 11a 中可以看出，蓝色点代表数据样本，红色线为线性回归模型拟合结果。通

过分析可发现，降水量与风速呈正相关关系，表明随着风速的增加，降水量也相应增加。在图 11b 中，蓝色点代表数据样本，黄色线为指数衰减拟合结果。除了降水量与风速呈正相关关系外，该图的数据点更密集，拟合效果更好。



(a) 降水量与风速的关系（线性回归模型）



(b) 降水量与风速的关系（指数衰减拟合）

图 11 降水量与风速的关系

降水量与距离的关系模型的评估结果如表 10 所示。

表 10 降水量与距离模型评估结果

模型类型	R^2	MSE
线性回归模型	0.0600	245.55
指数衰减模型	0.0842	0.3671
幂律衰减模型	0.0667	0.3741

同样的，我们使用散点图和回归线对线性模型和非线性模型进行可视化，图 12 所

示。从图 12a 中可以看出，绿色点代表数据样本，红色线为线性回归模型拟合结果。通过分析可发现，降水量与风速呈正相关关系，表明随着风速的增加，降水量也相应增加，但是数据点相对分散。在图 12b 中，绿色点代表数据样本，黄色线为指数衰减拟合结果。除了降水量与风速呈正相关关系外，该图的数据点更密集，拟合效果更好。

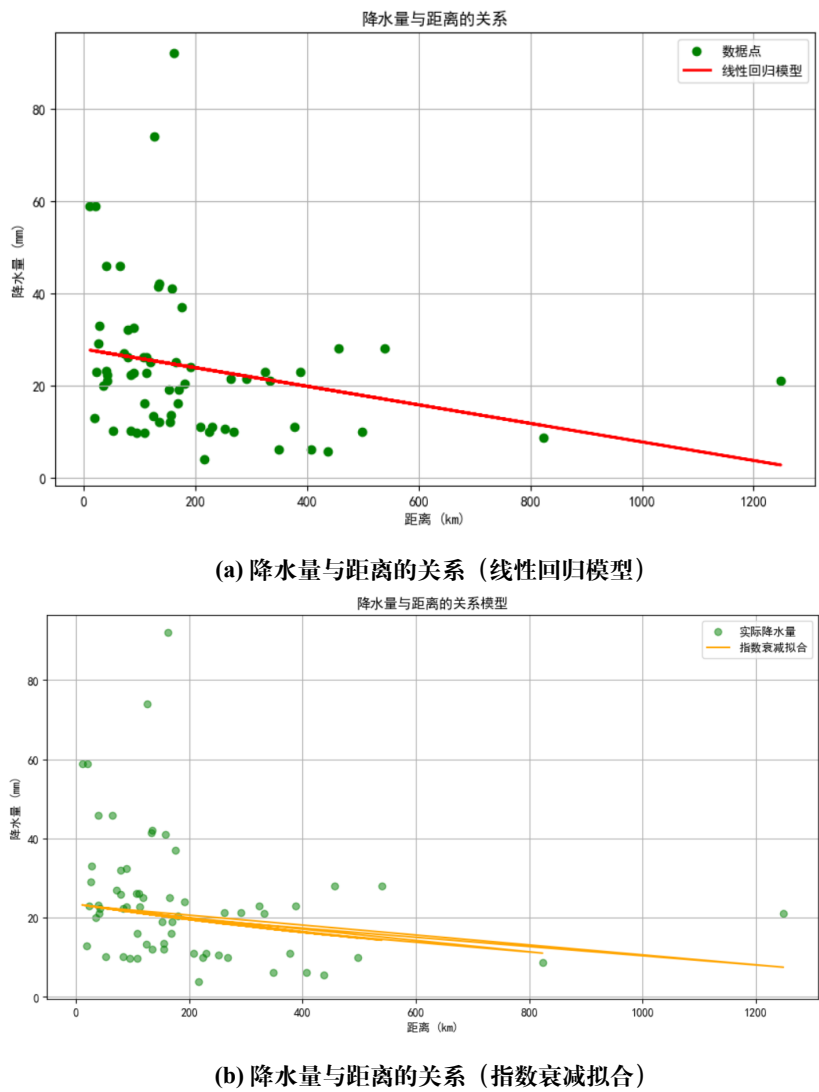


图 12 降水量与距离的关系

综上所述，本文建立了降水量与风速及距离的关系模型，并进行了模型评估。结果表明，指数衰减模型在降水量与风速及距离的关系建模中均优于幂律衰减模型。具体而言，降水量与风速的指数衰减模型和降水量与距离的指数衰减模型均显示出良好的拟合效果，为进一步的气象研究和预测提供了可靠依据。

八、模型的分析与检验

8.1 灵敏度分析

灵敏度分析旨在评估模型对输入参数变化的敏感程度。通过对关键参数进行微小扰动，可以观察到模型输出的变化，从而判断各参数对模型预测结果的影响程度。在本研究中，我们选择了几个主要参数进行灵敏度分析，包括：

- **风速**：作为影响台风路径和强度的重要因素，风速的变化对模型输出的影响尤为显著。通过调整风速的值，我们分析了其对台风预测路径的影响。
- **气压**：气压是影响台风强度的重要因素。灵敏度分析中，我们通过改变气压的值，观察其对台风强度预测的影响。
- **海面温度**：海面温度直接影响台风的生成和发展。我们通过改变海面温度来研究其对台风形成概率的影响。

分析结果表明，风速和气压的变化对模型预测结果影响较大，特别是在临界值附近，模型输出可能会有显著变化。这表明在模型建立过程中，应特别关注这些参数的准确性和可靠性。

8.2 误差分析

误差分析是对模型预测结果与实际观测值之间的差异进行定量评估。通过计算多种误差指标，我们能够全面评估模型的性能。常用的误差指标包括：

- **均方根误差 (RMSE)**：衡量预测值与实际值之间的均方根误差，反映模型预测的精度。

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (32)$$

- **平均绝对误差 (MAE)**：计算预测值与实际值之间的绝对误差的平均值，直观反映模型的准确性。

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (33)$$

- **R² 决定系数**：衡量模型解释观测值变异的能力。R² 值越接近 1，表示模型拟合效果越好。

通过对比模型的预测结果与实际数据，如表7、表9、表10，我们发现模型在大多数情况下能够较好地捕捉台风的路径和强度变化。然而，在极端天气事件的预测中，模型的表现可能受到影响，导致误差增大。因此，进一步优化模型，增强其在极端情况下的预测能力是未来的研究方向。

九、模型优缺点分析

9.1 DBSCAN 模型

- **能够识别任意形状的簇：**DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 是一种基于密度的聚类算法，其主要优点在于能够有效地识别任意形状的簇。这一特点使得 DBSCAN 在处理台风路径预测时，能够识别复杂的风速和降水量分布，适用于非线性分布的数据。对于不同类型的台风，DBSCAN 能够将相似的路径聚类，从而更好地理解 and 预测其运动规律。
- **对参数敏感：**DBSCAN 的主要缺点之一是对参数选择的敏感性。该算法需要设定两个关键参数：半径 (Eps) 和最小邻域点数 (MinPts)。这些参数的选择直接影响聚类结果的质量。在台风预测中，若参数设置不当，可能导致数据点被错误分类，影响对台风路径的正确识别与预测。因此，合理的参数调整是提高模型有效性的关键。

9.2 基于注意力机制的 CNN-LSTM 模型

- **长短期记忆与特征提取：**LSTM 网络能够有效捕捉时间序列数据中的长期依赖关系，特别适合处理台风路径预测等时序任务。同时，卷积神经网络 (CNN) 在输入数据中自动提取重要特征，从而提高模型的性能。注意力机制通过动态调整对不同时间步的关注程度，增强了模型对关键信息的捕捉能力，提升了预测的准确性。这对于台风预测至关重要，因为台风路径受多种因素影响，长短期记忆帮助模型考虑到历史路径和实时气象数据的关联。
- **计算复杂性与过拟合风险：**尽管该模型在捕捉复杂关系方面表现出色，但其结构相对复杂，计算资源需求较高，训练时间较长，尤其在处理高维数据时。此外，模型参数较多，可能导致在训练数据较少时出现过拟合问题，这使得模型在实际应用中的稳定性受到挑战。在台风预测中，若训练数据不足，模型可能无法有效泛化，影响实际预测效果。

9.3 指数衰减模型

- **简单易用与直观性强：**指数衰减模型结构简单，易于理解和实现，适合快速建模。通过简单的数学公式描述降水量与风速等因素的关系，便于研究者进行解释和应用。在台风研究中，该模型可快速建立降水量和风速随距离变化的基本关系，为研究台风影响范围提供初步估计。
- **线性假设与对异常值敏感：**尽管模型简单易用，但其假设降水量与风速的关系是线性的，这可能无法准确捕捉实际数据中的复杂非线性关系。在台风预测中，实际降水量与风速之间的关系往往受多种因素影响，存在较大的不确定性。此外，模型对数据中的异常值敏感，异常值的存在可能会显著影响整体预测性能，从而降低模型

的可靠性。在实际应用中, 需要结合其他模型进行验证与补充, 以提高预测的准确性。

参考文献

- [1] 崔炳唱, 黄鹏. 台风气候下低矮建筑屋面实测风压的分形特征[J]. 湖南大学学报 (自然科学版), 2024.
- [2] 百度网盘[EB/OL]. 2024. https://pan.baidu.com/s/1JZ_163qGoIDQkKqcDvT4Pw.
- [3] 杨真真, 李泓瑶, 杨可易, 等. 基于系统聚类算法与肘部准则的各省市居民消费水平聚类分析[J]. 中国科技信息, 2021.
- [4] 龙文佳, 张晓峰, 张链. 基于 k-means 和肘部法则的业务流程聚类方法[J]. 江汉大学学报 (自然科学版), 2020.
- [5] 于宗超, 陈孜孜, 文明, 等. 基于 VAE-Kmeans 算法的台区聚类精准画像技术[J]. 湖南电力, 2024.
- [6] 孙健玮, 刘玉龙. 基于层次聚类的图文检索模型研究[J]. 计算机测量与控制, 2024.
- [7] 廖剑, 孙瑜, 刘若愚, 等. 基于 DBSCAN 算法的高速公路车群行为特征与安全风险分析[J]. 智能计算机与应用, 2024.
- [8] 梁天成, 李凌. 计及碳交易与需求响应的多典型日综合能源系统优化模型[J]. 热能动力工程, 2024.
- [9] 王洁霖. 基于 Calinski-Harabaz 指标的无监督学习研究拓扑相变[Z]. [出版地不详]: 太原理工大学, 2021.
- [10] 姜宏维, 程雨, 段志. 基于 K 均值 +Davies—Bouldin 指数 +PCA 法的桥梁结构性能综合评估算法研究[J]. 公路, 2024.
- [11] 赵生智, 申胤宏, 周鹰槐. 基于斯皮尔曼相关系数的电网企业区间岗级任职资格信度研究[C]//中国电力企业管理创新实践 (2022 年). [出版地不详]: 云南电网有限责任公司, 2024.
- [12] 田琼, 欧阳政, 曲强, 等. X 射线荧光光谱结合主成分分析-线性判别分析对炼焦煤产地识别[J]. 冶金分析, 2024.
- [13] 王海川, 李蓉, 郭英飞. 基于随机森林算法在通风机故障检测中的应用[J]. 江西煤炭科技, 2024.

- [14] 杨靖, 焦童, 董宇娇, 等. 基于 XGBoost 的 COPD 患者肺癌发生预测模型的建立与评价[J]. 西安交通大学学报 (医学版), 2024.
- [15] 侯慧, 吴文杰, 魏瑞增, 等. 基于注意力机制的 CNN-LSTM-XGBoost 台风暴雨电力气象混合预测模型[J]. 智慧电力, 2024, 23.
- [16] 张流洋, 马义中, 任鸣鸣, 等. 基于均方根误差建模的多响应稳健参数设计[J]. 统计与决策, 2020.
- [17] 陈希孺. 一种非参数回归估计量的条件平均绝对误差的指数收敛速度[J]. 数学学报, 1987.
- [18] 程蕾. 具有时间效应的小区域估计及其均方误差的估计[Z]. [出版地不详]: 华东师范大学, 2023.
- [19] 高威, 钱程扬, 张琪, 等. 基于动态时间规整与轨迹点压缩的轨迹相似性算法[J]. 控制与信息技术, 2024.
- [20] 中国气象局[EB/OL]. 2024. <http://www.cma.gov.cn/>.
- [21] 星图云开放平台[EB/OL]. 2024. <https://open.geovisearth.com/service/resource>.
- [22] L B J, L R, S S. Real-time healthcare monitoring system through haversine distance calculation-based global positioning system[J]. International Journal of Medical Engineering and Informatics, 2024.
- [23] 中国气象局热带气旋资料[EB/OL]. 2024. <https://tcdata.typhoon.org.cn/rdqxfy.html>.

附录 A 文件列表

文件名	功能描述
Q1.py	问题一程序核心代码
Q2.py	问题二程序核心代码
Q3.py	问题三程序核心代码

表 11 文件列表

附录 B 代码

Q1.py

```
1 # 导入必要的库
2 import pandas as pd # 数据处理
3 import numpy as np # 数值计算
4 import matplotlib.pyplot as plt # 绘图
5 # import seaborn as sns # 数据可视化
6 from sklearn.model_selection import train_test_split # 数据集
   划分
7 from sklearn.ensemble import RandomForestClassifier # 随机森
   林分类器
8 from sklearn.metrics import classification_report,
   confusion_matrix # 模型评估
9 from sklearn.preprocessing import StandardScaler # 特征标准化
10
11 # 加载数据集
12 file_path = '1945-2023.xlsx'
13 data = pd.read_excel(file_path)
14
15 import pandas as pd
16
17
18 # 定义一个函数来转换时间格式
19 def convert_to_datetime(column):
20     return pd.to_datetime(data[column], errors='coerce')
```

```

21
22
23 # 将台风起始时间、结束时间和当前台风时间转换为日期时间格式
24 time_columns = ['台风起始时间', '台风结束时间', '当前台风时间'
25                 ]
26 for col in time_columns:
27     data[col] = convert_to_datetime(col)
28
29 # 计算持续时间（以天为单位），并添加为新列
30 data['台风持续时间'] = (data['台风结束时间'] - data['台风起始
31     时间']).dt.total_seconds() / (24 * 3600)
32
33 # 提取月份信息
34 data['月份'] = data['当前台风时间'].dt.month
35
36 # 删除含有缺失值的行
37 data_cleaned = data.dropna()
38
39 # 导出清理后的数据
40 data_cleaned.to_excel('台风数据_清理后.xlsx', index=False)
41
42 # 从清理后的数据中读取
43 data = pd.read_excel('台风数据_清理后.xlsx')
44
45 # 区分夏台风（6-8月）和秋台风（9-11月）
46 def categorize_season(month):
47     if 6 <= month <= 8:
48         return '夏台风'
49     elif 9 <= month <= 11:
50         return '秋台风'
51     return '其他'
52
53 data['季节'] = data['月份'].apply(categorize_season)

```

```

54
55 # 过滤出夏台风和秋台风的数据
56 data = data[data['季节'].isin(['夏台风', '秋台风'])].copy()
57
58
59 # 类别划分标准
60 # 对'台风强度'进行编码
61 def encode_typhoon_intensity(intensity):
62     mapping = {
63         '热带低压(TD)': '1',
64         '热带风暴(TS)': '2',
65         '强热带风暴(STS)': '3',
66         '台风(TY)': '4',
67         '强台风(STY)': '5',
68         '超强台风(Super TY)': '6'
69     }
70     return int(mapping.get(intensity, 0))
71
72
73 data['台风强度'] = data['台风强度'].apply(lambda x:
74     encode_typhoon_intensity(x) if pd.notna(x) else 0)
75
76 # 对'移动方向'进行编码
77 def encode_movement_direction(direction):
78     mapping = {'偏': '0', '北': '1', '西': '2', '东': '3', '南': '4'}
79     if pd.notna(direction):
80         return int(''.join(mapping[char] for char in direction
81             if char in mapping)) or 0
82     return 0
83
84 data['移动方向'] = data['移动方向'].apply(
85     encode_movement_direction)

```

```

85
86 # 导出处理后的数据到新的Excel文件
87 data.to_excel('台风数据_处理后_编码方式修改.xlsx', index=False
88             )
89
90 import pandas as pd
91 from sklearn.cluster import KMeans, DBSCAN,
92     AgglomerativeClustering
93 from sklearn.preprocessing import StandardScaler
94 from sklearn.metrics import silhouette_score,
95     calinski_harabasz_score, davies_bouldin_score
96
97 import matplotlib.pyplot as plt
98 import matplotlib.font_manager as fm
99 import warnings
100
101 file_path = '台风数据_处理后_编码方式修改.xlsx'
102 data = pd.read_excel(file_path)
103 # 忽略警告信息
104 warnings.filterwarnings('ignore', category=DeprecationWarning)
105
106 # 设置字体为支持中文的字体 (SimHei)
107 plt.rcParams['font.sans-serif'] = ['SimHei']
108 plt.rcParams['axes.unicode_minus'] = False # 用于正常显示负号
109
110 import matplotlib.pyplot as plt
111 from sklearn.preprocessing import StandardScaler
112 from sklearn.cluster import KMeans, AgglomerativeClustering,
113     DBSCAN
114 from sklearn.metrics import silhouette_score,
115     calinski_harabasz_score, davies_bouldin_score
116
117 # 选择用于聚类分析的特征：台风强度、等级、风速、气压
118 features = ['台风强度', '台风等级', '风速', '气压']
119 data_filtered = data.dropna(subset=features)
120

```

```

115 # 数据标准化
116 scaler = StandardScaler()
117 features_scaled = scaler.fit_transform(data_filtered[features
    ])
118
119 # 使用肘方法确定最佳聚类数目
120 sse = []
121 k_range = range(2, 11)
122 for k in k_range:
123     kmeans = KMeans(n_clusters=k, random_state=42)
124     kmeans.fit(features_scaled)
125     sse.append(kmeans.inertia_)
126
127 # 绘制肘方法图
128 plt.figure(figsize=(10, 6))
129 plt.plot(k_range, sse, marker='o')
130 plt.xlabel('聚类数目 (k)')
131 plt.ylabel('SSE (误差平方和)')
132 plt.title('肘方法确定最佳聚类数目')
133 plt.show()
134
135 # 根据肘方法选择合适的聚类数目 (可以根据显示的图形调整)
136 optimal_k = 6
137
138
139 # 定义一个函数来执行聚类并计算评价指标
140 def evaluate_clustering(algorithm, features_scaled, n_clusters
    =None):
141     if algorithm == 'KMeans':
142         model = KMeans(n_clusters=n_clusters, random_state=42)
143     elif algorithm == 'Agglomerative':
144         model = AgglomerativeClustering(n_clusters=n_clusters)
145     elif algorithm == 'DBSCAN':
146         model = DBSCAN(eps=0.5, min_samples=5)
147     else:

```

```

148         raise ValueError("未定义的聚类算法。")
149
150     labels = model.fit_predict(features_scaled)
151
152     results = {}
153     if algorithm != 'DBSCAN' or len(set(labels)) > 1: #
154         DBSCAN特殊处理
155         results['silhouette'] = silhouette_score(
156             features_scaled, labels)
157         results['calinski_harabasz'] = calinski_harabasz_score
158             (features_scaled, labels)
159         results['davies_bouldin'] = davies_bouldin_score(
160             features_scaled, labels)
161     else:
162         results = {'silhouette': -1, 'calinski_harabasz': -1,
163             'davies_bouldin': -1} # 只有一个类的情况
164
165     return results
166
167 # 使用多种聚类算法进行聚类并评价
168 algorithms = ['KMeans', 'Agglomerative']
169 results = {
170     '算法': [],
171     '轮廓系数(SC)': [],
172     'Calinski-Harabaz(CH)': [],
173     'Davies-Bouldin Index(DB)': []
174 }
175
176 for algo in algorithms:
177     metrics = evaluate_clustering(algo, features_scaled,
178         optimal_k)
179     results['算法'].append(algo)
180     results['轮廓系数(SC)'].append(metrics['silhouette'])
181     results['Calinski-Harabaz(CH)'].append(metrics['

```

```

    calinski_harabasz']])
177     results['Davies-Bouldin Index(DB)'].append(metrics['
    davies_bouldin'])
178
179 # 处理 DBSCAN
180 dbscan_metrics = evaluate_clustering('DBSCAN', features_scaled
    )
181 results['算法'].append('DBSCAN')
182 results['轮廓系数(SC)'].append(dbscan_metrics['silhouette'])
183 results['Calinski-Harabaz(CH)'].append(dbscan_metrics['
    calinski_harabasz'])
184 results['Davies-Bouldin Index(DB)'].append(dbscan_metrics['
    davies_bouldin'])
185
186 # 将聚类结果与评价指标进行比较
187 results_df = pd.DataFrame(results)
188 results_df
189
190 import pandas as pd
191 from sklearn.preprocessing import StandardScaler
192 from sklearn.decomposition import PCA
193 from sklearn.cluster import DBSCAN
194 import matplotlib.pyplot as plt
195 import warnings
196
197
198 def load_data(file_path):
199     """加载Excel数据"""
200     return pd.read_excel(file_path)
201
202
203 def preprocess_data(data, features):
204     """数据预处理，包括去除缺失值和标准化"""
205     data_filtered = data.dropna(subset=features)
206     scaler = StandardScaler()

```

```

207     features_scaled = scaler.fit_transform(data_filtered[
features])
208     return data_filtered, features_scaled
209
210
211 def cluster_data(features_scaled):
212     """使用DBSCAN进行聚类"""
213     dbscan = DBSCAN(eps=0.5, min_samples=5)
214     return dbscan.fit_predict(features_scaled)
215
216
217 def add_classification(data, labels):
218     """根据聚类结果添加台风类别"""
219     data['聚类类别'] = labels
220     data['台风类别'] = data.apply(
221         lambda row: f"{row['季节']}{chr(65 + row['聚类类别'])}"
222         " if row['聚类类别'] >= 0 else "未分类", axis=1
223     )
224     return data
225
226
227 def perform_pca(features_scaled):
228     """使用PCA降维至2D"""
229     pca = PCA(n_components=2)
230     return pca.fit_transform(features_scaled)
231
232
233 def plot_results(pca_features_2d, data_filtered):
234     """绘制台风类别的2D聚类可视化"""
235     plt.figure(figsize=(12, 8))
236     unique_labels = data_filtered['台风类别'].unique()
237     colors = plt.cm.get_cmap('Set2', len(unique_labels))
238
239     for i, label in enumerate(unique_labels):
240         class_data = pca_features_2d[data_filtered['台风类别']

```

```

240     == label]
241     plt.scatter(class_data[:, 0], class_data[:, 1],
242                  color=colors(i), label=label, s=60, alpha
243                  =0.8, edgecolor='w')
244
245     # 设置中文字体，避免负号显示问题
246     plt.rcParams['font.sans-serif'] = ['SimHei']
247     plt.rcParams['axes.unicode_minus'] = False
248
249     plt.grid(True)
250     plt.title('台风类别的2D聚类可视化', fontsize=16)
251     plt.xlabel('PCA Component 1', fontsize=12)
252     plt.ylabel('PCA Component 2', fontsize=12)
253     plt.legend(title='台风类别', title_fontsize='13', loc='
254     best')
255     plt.xlim((-4, 6))
256     plt.ylim((-1, 3))
257     plt.show()
258
259 def save_results(data, output_prefix, max_rows_per_file
260                 =500000):
261     """将结果分多段导出为Excel文件"""
262     num_parts = len(data) // max_rows_per_file + 1
263
264     for i in range(num_parts):
265         start_row = i * max_rows_per_file
266         end_row = min((i + 1) * max_rows_per_file, len(data))
267         output_file_path = f'{output_prefix}_part{i + 1}.xlsx'
268         data.iloc[start_row:end_row].to_excel(output_file_path
269         , index=False)
270         print(f"处理后的数据已导出到 {output_file_path}")
271
272     # 主程序

```

```

270
271 def main():
272     warnings.filterwarnings('ignore', category=
DeprecationWarning)
273
274     file_path = '台风数据_处理后_编码方式修改.xlsx'
275     features = ['台风强度', '台风等级', '风速', '气压']
276
277     # 1. 加载数据
278     data = load_data(file_path)
279
280     # 2. 数据预处理
281     data_filtered, features_scaled = preprocess_data(data,
features)
282
283     # 3. 聚类
284     cluster_labels = cluster_data(features_scaled)
285
286     # 4. 加入分类信息
287     data_filtered = add_classification(data_filtered,
cluster_labels)
288
289     # 5. PCA降维
290     pca_features_2d = perform_pca(features_scaled)
291
292     # 6. 可视化结果
293     plot_results(pca_features_2d, data_filtered)
294
295     # 7. 保存最终结果
296     data['台风类别'] = data_filtered['台风类别']
297     data_final = data.drop(columns=['季节'])
298     save_results(data_final, '台风分类结果_最终')
299
300
301 if __name__ == "__main__":

```

```

302     main()
303
304 import pandas as pd
305 import matplotlib.pyplot as plt
306 from mpl_toolkits.basemap import Basemap
307 import warnings
308
309 # 忽略警告信息
310 warnings.filterwarnings('ignore', category=DeprecationWarning)
311
312 # 设置字体为支持中文的字体 (SimHei)
313 plt.rcParams['font.sans-serif'] = ['SimHei']
314 plt.rcParams['axes.unicode_minus'] = False # 用于正常显示负号
315
316
317 def load_data(file_path):
318     """加载Excel数据并返回DataFrame"""
319     try:
320         data = pd.read_excel(file_path)
321         return data
322     except Exception as e:
323         print(f"加载数据时出错: {e}")
324         return None
325
326
327 def plot_typhoon_map(data):
328     """在地图上绘制台风分类"""
329     lon = data['经度']
330     lat = data['纬度']
331     category = data['台风类别']
332
333     unique_categories = category.unique()
334     colors = plt.cm.tab10(range(len(unique_categories)))
335
336     plt.figure(figsize=(14, 10))

```

```

337     m = Basemap(projection='cyl', resolution='l', llcrnrlat
=-20, urcrnrlat=45, llcrnrlon=100, urcrnrlon=180)
338     m.drawcoastlines()
339     m.drawcountries()
340     m.drawmapboundary(fill_color='lightblue')
341     m.fillcontinents(color='lightgrey', lake_color='lightblue'
)
342
343     for i, cat in enumerate(unique_categories):
344         indices = category == cat
345         m.scatter(lon[indices], lat[indices], latlon=True,
label=cat, color=colors[i], s=10, alpha=0.7)
346
347     plt.legend(title='台风类别', loc='upper right')
348     plt.title('台风类别经纬度可视化')
349     plt.xlabel('经度')
350     plt.ylabel('纬度')
351     plt.show()
352
353
354 def main():
355     file_path = '台风分类结果_最终_part1.xlsx'
356     data = load_data(file_path)
357
358     if data is not None:
359         plot_typhoon_map(data)
360
361
362 if __name__ == "__main__":
363     main()

```

Q2.py

```

1 # 导入必要的库
2 import pandas as pd # 数据处理
3 import numpy as np # 数值计算

```

```

4 import matplotlib.pyplot as plt # 绘图
5 # import seaborn as sns # 数据可视化
6 from sklearn.model_selection import train_test_split # 数据集
   划分
7 from sklearn.ensemble import RandomForestClassifier # 随机森
   林分类器
8 from sklearn.metrics import classification_report,
   confusion_matrix # 模型评估
9 from sklearn.preprocessing import StandardScaler # 特征标准化
10
11 # 加载数据集
12 file_path = '1945-2023.xlsx'
13 data = pd.read_excel(file_path)
14
15 import pandas as pd
16
17
18 # 定义一个函数来转换时间格式
19 def convert_to_datetime(column):
20     return pd.to_datetime(data[column], errors='coerce')
21
22
23 # 将台风起始时间、结束时间和当前台风时间转换为日期时间格式
24 time_columns = ['台风起始时间', '台风结束时间', '当前台风时间'
   ]
25 for col in time_columns:
26     data[col] = convert_to_datetime(col)
27
28 # 计算持续时间（以天为单位），并添加为新列
29 data['台风持续时间'] = (data['台风结束时间'] - data['台风起始
   时间']).dt.total_seconds() / (24 * 3600)
30
31 # 提取月份信息
32 data['月份'] = data['当前台风时间'].dt.month
33

```

```

34 # 删除含有缺失值的行
35 data_cleaned = data.dropna()
36
37 # 导出清理后的数据
38 data_cleaned.to_excel('台风数据_清理后.xlsx', index=False)
39
40 # 从清理后的数据中读取
41 data = pd.read_excel('台风数据_清理后.xlsx')
42
43
44 # 区分夏台风（6-8月）和秋台风（9-11月）
45 def categorize_season(month):
46     if 6 <= month <= 8:
47         return '夏台风'
48     elif 9 <= month <= 11:
49         return '秋台风'
50     return '其他'
51
52
53 data['季节'] = data['月份'].apply(categorize_season)
54
55 # 过滤出夏台风和秋台风的数据
56 data = data[data['季节'].isin(['夏台风', '秋台风'])].copy()
57
58
59 # 类别划分标准
60 # 对'台风强度'进行编码
61 def encode_typhoon_intensity(intensity):
62     mapping = {
63         '热带低压(TD)': '1',
64         '热带风暴(TS)': '2',
65         '强热带风暴(STS)': '3',
66         '台风(TY)': '4',
67         '强台风(STY)': '5',
68         '超强台风(Super TY)': '6'

```

```

69     }
70     return int(mapping.get(intensity, 0))
71
72
73 data['台风强度'] = data['台风强度'].apply(lambda x:
74     encode_typhoon_intensity(x) if pd.notna(x) else 0)
75
76 # 对'移动方向'进行编码
77 def encode_movement_direction(direction):
78     mapping = {'偏': '0', '北': '1', '西': '2', '东': '3', '南': '4'}
79     if pd.notna(direction):
80         return int(''.join(mapping[char] for char in direction
81             if char in mapping)) or 0
82     return 0
83
84 data['移动方向'] = data['移动方向'].apply(
85     encode_movement_direction)
86
87 # 导出处理后的数据到新的Excel文件
88 data.to_excel('台风数据_处理后_编码方式修改.xlsx', index=False)
89
90 import pandas as pd
91 from sklearn.cluster import KMeans, DBSCAN,
92     AgglomerativeClustering
93 from sklearn.preprocessing import StandardScaler
94 from sklearn.metrics import silhouette_score,
95     calinski_harabasz_score, davies_bouldin_score
96 import matplotlib.pyplot as plt
97 import matplotlib.font_manager as fm
98 import warnings

```

```

97 file_path = '台风数据_处理后_编码方式修改.xlsx'
98 data = pd.read_excel(file_path)
99 # 忽略警告信息
100 warnings.filterwarnings('ignore', category=DeprecationWarning)
101
102 # 设置字体为支持中文的字体 (SimHei)
103 plt.rcParams['font.sans-serif'] = ['SimHei']
104 plt.rcParams['axes.unicode_minus'] = False # 用于正常显示负号
105
106 import matplotlib.pyplot as plt
107 from sklearn.preprocessing import StandardScaler
108 from sklearn.cluster import KMeans, AgglomerativeClustering,
    DBSCAN
109 from sklearn.metrics import silhouette_score,
    calinski_harabasz_score, davies_bouldin_score
110
111 # 选择用于聚类分析的特征：台风强度、等级、风速、气压
112 features = ['台风强度', '台风等级', '风速', '气压']
113 data_filtered = data.dropna(subset=features)
114
115 # 数据标准化
116 scaler = StandardScaler()
117 features_scaled = scaler.fit_transform(data_filtered[features
    ])
118
119 # 使用肘方法确定最佳聚类数目
120 sse = []
121 k_range = range(2, 11)
122 for k in k_range:
123     kmeans = KMeans(n_clusters=k, random_state=42)
124     kmeans.fit(features_scaled)
125     sse.append(kmeans.inertia_)
126
127 # 绘制肘方法图
128 plt.figure(figsize=(10, 6))

```

```

129 plt.plot(k_range, sse, marker='o')
130 plt.xlabel('聚类数目 (k)')
131 plt.ylabel('SSE (误差平方和)')
132 plt.title('肘方法确定最佳聚类数目')
133 plt.show()
134
135 # 根据肘方法选择合适的聚类数目 (可以根据显示的图形调整)
136 optimal_k = 6
137
138
139 # 定义一个函数来执行聚类并计算评价指标
140 def evaluate_clustering(algorithm, features_scaled, n_clusters
    =None):
141     if algorithm == 'KMeans':
142         model = KMeans(n_clusters=n_clusters, random_state=42)
143     elif algorithm == 'Agglomerative':
144         model = AgglomerativeClustering(n_clusters=n_clusters)
145     elif algorithm == 'DBSCAN':
146         model = DBSCAN(eps=0.5, min_samples=5)
147     else:
148         raise ValueError("未定义的聚类算法。")
149
150     labels = model.fit_predict(features_scaled)
151
152     results = {}
153     if algorithm != 'DBSCAN' or len(set(labels)) > 1: #
154         DBSCAN特殊处理
155         results['silhouette'] = silhouette_score(
156             features_scaled, labels)
157         results['calinski_harabasz'] = calinski_harabasz_score
158             (features_scaled, labels)
159         results['davies_bouldin'] = davies_bouldin_score(
160             features_scaled, labels)
161     else:
162         results = {'silhouette': -1, 'calinski_harabasz': -1,

```

```

    'davies_bouldin': -1} # 只有一个类的情况
159
160     return results
161
162
163 # 使用多种聚类算法进行聚类并评价
164 algorithms = ['KMeans', 'Agglomerative']
165 results = {
166     '算法': [],
167     '轮廓系数(SC)': [],
168     'Calinski-Harabaz(CH)': [],
169     'Davies-Bouldin Index(DB)': []
170 }
171
172 for algo in algorithms:
173     metrics = evaluate_clustering(algo, features_scaled,
174                                   optimal_k)
175     results['算法'].append(algo)
176     results['轮廓系数(SC)'].append(metrics['silhouette'])
177     results['Calinski-Harabaz(CH)'].append(metrics['
178 calinski_harabasz'])
179     results['Davies-Bouldin Index(DB)'].append(metrics['
180 davies_bouldin'])
181
182 # 处理 DBSCAN
183 dbscan_metrics = evaluate_clustering('DBSCAN', features_scaled
184 )
185 results['算法'].append('DBSCAN')
186 results['轮廓系数(SC)'].append(dbscan_metrics['silhouette'])
187 results['Calinski-Harabaz(CH)'].append(dbscan_metrics['
188 calinski_harabasz'])
189 results['Davies-Bouldin Index(DB)'].append(dbscan_metrics['
190 davies_bouldin'])
191
192 # 将聚类结果与评价指标进行比较

```

```

187 results_df = pd.DataFrame(results)
188 results_df
189
190 import pandas as pd
191 from sklearn.preprocessing import StandardScaler
192 from sklearn.decomposition import PCA
193 from sklearn.cluster import DBSCAN
194 import matplotlib.pyplot as plt
195 import warnings
196
197
198 def load_data(file_path):
199     """加载Excel数据"""
200     return pd.read_excel(file_path)
201
202
203 def preprocess_data(data, features):
204     """数据预处理，包括去除缺失值和标准化"""
205     data_filtered = data.dropna(subset=features)
206     scaler = StandardScaler()
207     features_scaled = scaler.fit_transform(data_filtered[
features])
208     return data_filtered, features_scaled
209
210
211 def cluster_data(features_scaled):
212     """使用DBSCAN进行聚类"""
213     dbscan = DBSCAN(eps=0.5, min_samples=5)
214     return dbscan.fit_predict(features_scaled)
215
216
217 def add_classification(data, labels):
218     """根据聚类结果添加台风类别"""
219     data['聚类类别'] = labels
220     data['台风类别'] = data.apply(

```

```

221         lambda row: f"{row['季节']}{chr(65 + row['聚类类别'])}"
222     " if row['聚类类别'] >= 0 else "未分类", axis=1
223 )
224 return data
225
226 def perform_pca(features_scaled):
227     """使用PCA降维至2D"""
228     pca = PCA(n_components=2)
229     return pca.fit_transform(features_scaled)
230
231
232 def plot_results(pca_features_2d, data_filtered):
233     """绘制台风类别的2D聚类可视化"""
234     plt.figure(figsize=(12, 8))
235     unique_labels = data_filtered['台风类别'].unique()
236     colors = plt.cm.get_cmap('Set2', len(unique_labels))
237
238     for i, label in enumerate(unique_labels):
239         class_data = pca_features_2d[data_filtered['台风类别']
240 == label]
241         plt.scatter(class_data[:, 0], class_data[:, 1],
242                     color=colors(i), label=label, s=60, alpha
243 =0.8, edgecolor='w')
244
245     # 设置中文字体，避免负号显示问题
246     plt.rcParams['font.sans-serif'] = ['SimHei']
247     plt.rcParams['axes.unicode_minus'] = False
248
249     plt.grid(True)
250     plt.title('台风类别的2D聚类可视化', fontsize=16)
251     plt.xlabel('PCA Component 1', fontsize=12)
252     plt.ylabel('PCA Component 2', fontsize=12)
253     plt.legend(title='台风类别', title_fontsize='13', loc='
best')

```

```

252     plt.xlim((-4, 6))
253     plt.ylim((-1, 3))
254     plt.show()
255
256
257 def save_results(data, output_prefix, max_rows_per_file
    =500000):
258     """将结果分多段导出为Excel文件"""
259     num_parts = len(data) // max_rows_per_file + 1
260
261     for i in range(num_parts):
262         start_row = i * max_rows_per_file
263         end_row = min((i + 1) * max_rows_per_file, len(data))
264         output_file_path = f'{output_prefix}_part{i + 1}.xlsx'
265         data.iloc[start_row:end_row].to_excel(output_file_path
    , index=False)
266         print(f"处理后的数据已导出到 {output_file_path}")
267
268     # 主程序
269
270
271 def main():
272     warnings.filterwarnings('ignore', category=
    DeprecationWarning)
273
274     file_path = '台风数据_处理后_编码方式修改.xlsx'
275     features = ['台风强度', '台风等级', '风速', '气压']
276
277     # 1. 加载数据
278     data = load_data(file_path)
279
280     # 2. 数据预处理
281     data_filtered, features_scaled = preprocess_data(data,
    features)
282

```

```

283     # 3. 聚类
284     cluster_labels = cluster_data(features_scaled)
285
286     # 4. 加入分类信息
287     data_filtered = add_classification(data_filtered,
cluster_labels)
288
289     # 5. PCA降维
290     pca_features_2d = perform_pca(features_scaled)
291
292     # 6. 可视化结果
293     plot_results(pca_features_2d, data_filtered)
294
295     # 7. 保存最终结果
296     data['台风类别'] = data_filtered['台风类别']
297     data_final = data.drop(columns=['季节'])
298     save_results(data_final, '台风分类结果_最终')
299
300
301 if __name__ == "__main__":
302     main()
303
304 import pandas as pd
305 import matplotlib.pyplot as plt
306 from mpl_toolkits.basemap import Basemap
307 import warnings
308
309 # 忽略警告信息
310 warnings.filterwarnings('ignore', category=DeprecationWarning)
311
312 # 设置字体为支持中文的字体 (SimHei)
313 plt.rcParams['font.sans-serif'] = ['SimHei']
314 plt.rcParams['axes.unicode_minus'] = False # 用于正常显示负号
315
316

```

```

317 def load_data(file_path):
318     """加载Excel数据并返回DataFrame"""
319     try:
320         data = pd.read_excel(file_path)
321         return data
322     except Exception as e:
323         print(f"加载数据时出错: {e}")
324         return None
325
326
327 def plot_typhoon_map(data):
328     """在地图上绘制台风分类"""
329     lon = data['经度']
330     lat = data['纬度']
331     category = data['台风类别']
332
333     unique_categories = category.unique()
334     colors = plt.cm.tab10(range(len(unique_categories)))
335
336     plt.figure(figsize=(14, 10))
337     m = Basemap(projection='cyl', resolution='l', llcrnrlat
=-20, urcrnrlat=45, llcrnrlon=100, urcrnrlon=180)
338     m.drawcoastlines()
339     m.drawcountries()
340     m.drawmapboundary(fill_color='lightblue')
341     m.fillcontinents(color='lightgrey', lake_color='lightblue'
)
342
343     for i, cat in enumerate(unique_categories):
344         indices = category == cat
345         m.scatter(lon[indices], lat[indices], latlon=True,
label=cat, color=colors[i], s=10, alpha=0.7)
346
347     plt.legend(title='台风类别', loc='upper right')
348     plt.title('台风类别经纬度可视化')

```

```

349     plt.xlabel('经度')
350     plt.ylabel('纬度')
351     plt.show()
352
353
354 def main():
355     file_path = '台风分类结果_最终_part1.xlsx'
356     data = load_data(file_path)
357
358     if data is not None:
359         plot_typhoon_map(data)
360
361
362 if __name__ == "__main__":
363     main()

```

Q3.py

```

1  import pandas as pd
2  from datetime import datetime
3
4  # 读取数据集
5  station_df = pd.read_csv('China_Sta.csv')
6  precipitation_df = pd.read_csv('1951-2018
   _MaxHourlyPrecipitation.csv', header=None)
7  typhoon_df = pd.read_excel('typhoon.xlsx')
8
9  # 为降水数据集添加列名
10 precipitation_columns = [
11     'tropical_cyclone_serial', 'tropical_cyclone_number', '
   station_number',
12     'max_rain_intensity_count', 'max_hourly_precipitation',
13     'first_max_rain_time', 'second_max_rain_time',
14     'third_max_rain_time', 'fourth_max_rain_time',
15     'fifth_max_rain_time', 'extra' # 额外一列
16 ]

```

```

17 precipitation_df.columns = precipitation_columns
18
19
20 # 定义函数将度分秒转换为十进制度
21 def dms_to_decimal(dms_str):
22     parts = dms_str.strip().split()
23     degrees, minutes, seconds = map(int, parts[:-1])
24     direction = parts[-1]
25     decimal = degrees + minutes / 60 + seconds / 3600
26     return decimal if direction in ['N', 'E'] else -decimal
27
28
29 # 将探测器的经纬度转换为十进制度
30 station_df['lat'] = station_df['lat'].apply(dms_to_decimal)
31 station_df['long'] = station_df['long'].apply(dms_to_decimal)
32
33
34 # 查找附近的探测器
35 def find_nearby_stations(lng, lat, station_df, tolerance=0.5):
36     return station_df[
37         (station_df['long'].between(lng - tolerance, lng +
38         tolerance)) &
39         (station_df['lat'].between(lat - tolerance, lat +
40         tolerance))
41     ]
42
43 # 解析时间范围并检查台风时间是否在降雨时间内
44 def is_typhoon_time_in_range(rain_times, typhoon_time):
45     for time in rain_times:
46         if isinstance(time, str) and time:
47             time_range = time.split('-')
48             if len(time_range) == 2: # 确保有起止时间
49                 start_time_str, end_time_str = map(str.strip,
50                 time_range)

```

```

49         try:
50             start_time = datetime.strptime(
start_time_str, '%Y%m%d%H%M')
51             end_time = datetime.strptime(end_time_str,
'%Y%m%d%H%M')
52             if start_time <= typhoon_time <= end_time:
53                 return True
54         except ValueError as e:
55             print(f"时间解析错误: {e} | start_time: {
start_time_str}, end_time: {end_time_str}")
56             return False
57
58
59 # 合并数据
60 results = []
61
62 for _, typhoon in typhoon_df.iterrows():
63     lng, lat = typhoon['lng'], typhoon['lat']
64     nearby_stations = find_nearby_stations(lng, lat,
station_df)
65
66     for _, station in nearby_stations.iterrows():
67         station_id = station['StationID']
68         rain_data = precipitation_df[precipitation_df['
station_number'] == station_id]
69
70         typhoon_time = datetime.strptime(typhoon['time'], '%Y
-%m-%dT%H:%M:%S')
71
72         for _, rain in rain_data.iterrows():
73             rain_times = [rain[i] for i in range(5, 10)] # 降
雨时间字段
74
75             if is_typhoon_time_in_range(rain_times,
typhoon_time):

```

```

76         result = {
77             'tropical_cyclone_serial': rain['
tropical_cyclone_serial'],
78             'tropical_cyclone_number': rain['
tropical_cyclone_number'],
79             'station_number': station_id,
80             'Typhoon Time': typhoon['time'],
81             '降水量': rain['max_hourly_precipitation'
],
82             '台风中文名称': typhoon['name'],
83             '台风强度': typhoon['strong'],
84             '风速等级': typhoon['power'],
85             '风速': typhoon['speed'],
86             '移动方向': typhoon['move_dir'],
87             '移动速度': typhoon['move_speed'],
88             '台风气压': typhoon['pressure'],
89             '经度': typhoon['lng'],
90             '纬度': typhoon['lat'],
91             '地点': typhoon['land'],
92         }
93         results.append(result)
94
95         # 创建结果DataFrame并保存
96     results_df = pd.DataFrame(results)
97     results_df.to_excel('降雨量_with_typhoon_features.xlsx', index
=False)
98
99     import pandas as pd
100    import numpy as np
101    import matplotlib.pyplot as plt
102    from sklearn.linear_model import LinearRegression
103    from sklearn.metrics import mean_squared_error, r2_score
104    import ast
105    import math
106

```

```

107 # 读取数据
108 df = pd.read_excel('降雨量_with_typhoon_features.xlsx')
109
110 # 提取新的三列
111 df['登陆时间'] = df['地点'].apply(lambda x: ast.literal_eval(x)
    )[0]['land_time'])
112 df['登陆经度'] = df['地点'].apply(lambda x: ast.literal_eval(x)
    )[0]['lng'])
113 df['登陆纬度'] = df['地点'].apply(lambda x: ast.literal_eval(x)
    )[0]['lat'])
114
115 # 打印结果
116 print(df[['登陆时间', '登陆经度', '登陆纬度']])
117
118
119 # 定义帮助函数以建立模型并评估
120 def create_and_evaluate_model(X, y, model_name):
121     model = LinearRegression()
122     model.fit(X, y)
123
124     # 预测和计算评价指标
125     y_pred = model.predict(X)
126     mse = mean_squared_error(y, y_pred)
127     r2 = r2_score(y, y_pred)
128
129     # 输出评价指标
130     print(f'{model_name} 的线性回归模型:')
131     print(f'MSE: {mse:.2f}')
132     print(f'R2: {r2:.2f}')
133
134     return model, y_pred
135
136
137 # 定义帮助函数以可视化
138 def plot_relationship(x, y, y_pred, x_label, y_label, title,

```

```

    color='blue'):
139     plt.figure(figsize=(10, 6))
140     plt.scatter(x, y, color=color, label='数据点')
141     plt.plot(x, y_pred, color='red', linewidth=2, label='线性
回归模型')
142     plt.xlabel(x_label)
143     plt.ylabel(y_label)
144     plt.title(title)
145     plt.legend()
146     plt.grid(True)
147     plt.show()
148
149
150 # 建立降水量与风速的关系模型
151 X_wind = df[['风速']]
152 y_rainfall = df['降水量']
153
154 rain_wind_model, y_pred_wind = create_and_evaluate_model(
    X_wind, y_rainfall, "降水量与风速")
155 plot_relationship(df['风速'], df['降水量'], y_pred_wind, '风速
(m/s)', '降水量 (mm)', '降水量与风速的关系')
156
157
158 # 定义Haversine公式计算距离
159 def haversine(lon1, lat1, lon2, lat2):
160     """
161     计算两个经纬度之间的距离（公里）
162     """
163     R = 6371 # 地球半径，单位公里
164     phi1 = math.radians(lat1)
165     phi2 = math.radians(lat2)
166     delta_phi = math.radians(lat2 - lat1)
167     delta_lambda = math.radians(lon2 - lon1)
168
169     a = math.sin(delta_phi / 2) ** 2 + \

```

```

170         math.cos(phi1) * math.cos(phi2) * math.sin(
171         delta_lambda / 2) ** 2
172     c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
173     distance = R * c
174     return distance
175
176
177 # 计算每条记录与参考点的距离
178 df['距离'] = df.apply(lambda row: haversine(row['登陆经度'],
179         row['登陆纬度'], row['经度'], row['纬度']), axis=1)
180
181 # 建立降水量与距离的关系模型
182 X_distance = df[['距离']]
183 y_rain_distance = df['降水量']
184
185 rain_distance_model, y_pred_distance =
186     create_and_evaluate_model(X_distance, y_rain_distance, "降水
187     量与距离")
188
189 plot_relationship(df['距离'], df['降水量'], y_pred_distance, '
190     距离 (km)', '降水量 (mm)', '降水量与距离的关系',
191     color='green')

```